

MOBILE TECHNOLOGIES AND PROGRAMMING

LAB5 – Android Programming, part I

1. Fragments and Dynamic UI

Create an Android application using:

- **Jetpack Compose**
- **Fragment**
- **FragmentManager**

The application should contain:

- a fragment container placed in the upper part of the screen,
- a button labeled **Fragment 1**,
- a button labeled **Fragment 2**.

After clicking the **Fragment 1** button, the application should display the first fragment.

After clicking the **Fragment 2** button, the currently displayed fragment should be replaced with the second fragment.

Fragment switching must be implemented dynamically using:

- `FragmentManager`
- `FragmentTransaction`

Each fragment must display its user interface using **Jetpack Compose**.

Requirements

The application should:

- use Compose-based UI,
- dynamically replace fragments during runtime,
- contain buttons for switching between fragments.

Each fragment should:

- display different content,
- use Material 3 Compose components,
- contain at least:
 - `Text`
 - `Column` **OR** `Row`.



Fig. 1

As a response to the task, please provide the code from all files you have modified or created in order to complete the task. Ensure not to include the entire project in your submission.

2. Adaptive UI and Language Resources with Jetpack Compose

Extend the application created in Task 1.

Modify the project so that the application adapts to:

- screen orientation,
- screen width,
- system language.

The application must:

- display a different layout in portrait and landscape orientation,
- support at least two languages, for example English and Polish
- load all visible text from string resources.

Orientation support

Implement adaptive UI behavior using:

- `LocalConfiguration`
- conditional composables.

Example:

```
if (configuration.orientation == Configuration.ORIENTATION_LANDSCAPE) {  
    LandscapeLayout()  
} else {  
    PortraitLayout()  
}
```

The portrait and landscape layouts should visibly differ from each other.

Language resources

Create separate language resource files:

- res/values/strings.xml
- res/values-pl/strings.xml
- res/values-en/strings.xml

The application should automatically display the correct language depending on the device settings.

Do not hardcode visible text directly inside composables.

Use:

```
stringResource()
```

to load string resources.

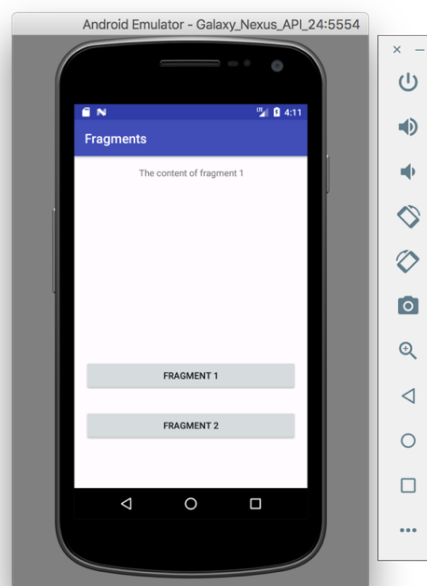


Fig. 2

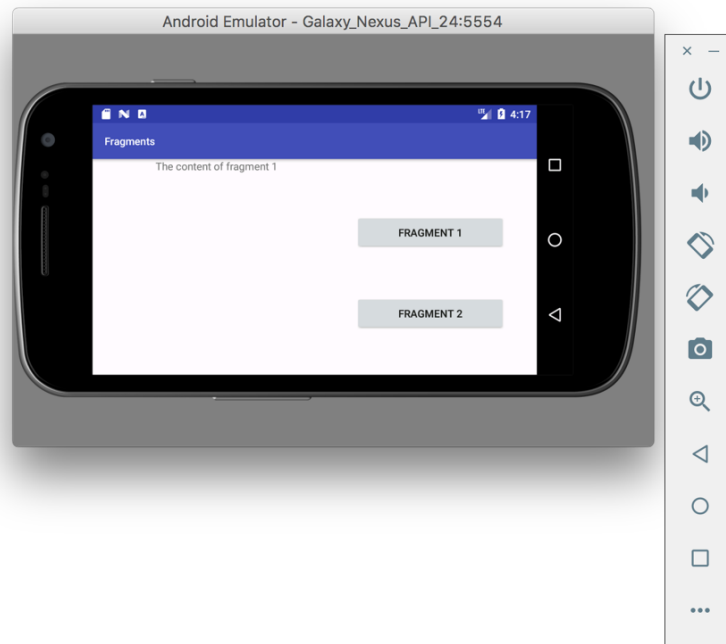


Fig. 3

As a response to the task, please provide the code from all files you have modified or created in order to complete the task. Ensure not to include the entire project in your submission.